Reproducible Data Analysis

Nikhil Singh Mangat

2026-04-16

1 Busiest Airports Analysis

This section examines annual passenger traffic across six of the world's busiest airports from 2020 to 2024. The three required airports (Hartsfield-Jackson Atlanta, Frankfurt, and Beijing Daxing) are supplemented by Dubai International (DXB), London Heathrow (LHR), and Dallas Fort Worth (DFW), providing a geographically diverse sample spanning North America, Europe, the Middle East, and Asia.

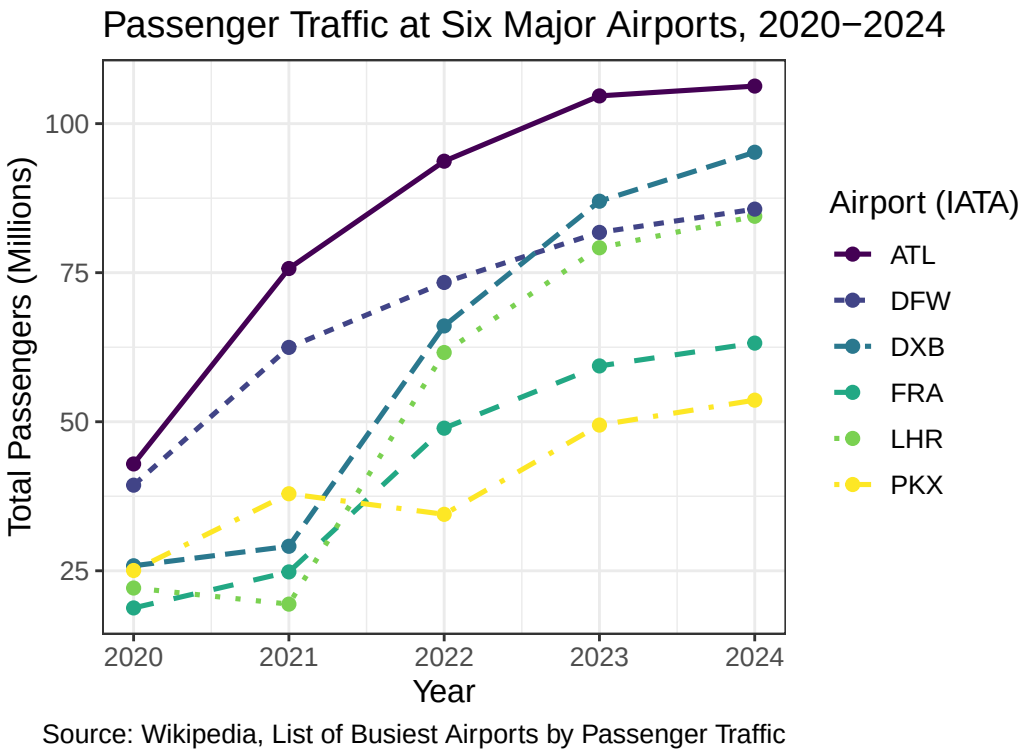
1.1 Table

Table 1: Annual Passenger Traffic (Millions) for Six of the World's Busiest Airports, 2020-2024 (Wikipedia contributors, 2026)

Airport (IATA)	2020	2021	2022	2023	2024
ATL	42.92	75.70	93.70	104.65	106.30
DXB	25.84	29.11	66.07	86.99	95.20
LHR	22.11	19.41	61.61	79.18	84.46
DFW	39.36	62.47	73.36	81.76	85.66
FRA	18.77	24.81	48.92	59.36	63.19
PKX	25.05	37.92	34.47	49.44	53.62

1.2 Plot

Figure 1: Line plot showing passenger traffic trends at six major airports from 2020 to 2024, with recovery following pandemic-era lows (Wikipedia contributors, 2026).



1.3 Narrative

When you look at the table and Figure 1, it is pretty obvious how much COVID hit air travel and how things have been recovering since. All six airports saw a big drop in 2020. Airports in Europe like Frankfurt and London Heathrow were hit especially hard, both dropping below 25 million passengers.

Atlanta did not fall as much compared to the others, which probably has to do with how strong domestic travel is in the U.S. Since domestic flights came back faster than international ones, ATL recovered quicker too. By 2024, it is back over 106 million passengers and still the busiest airport.

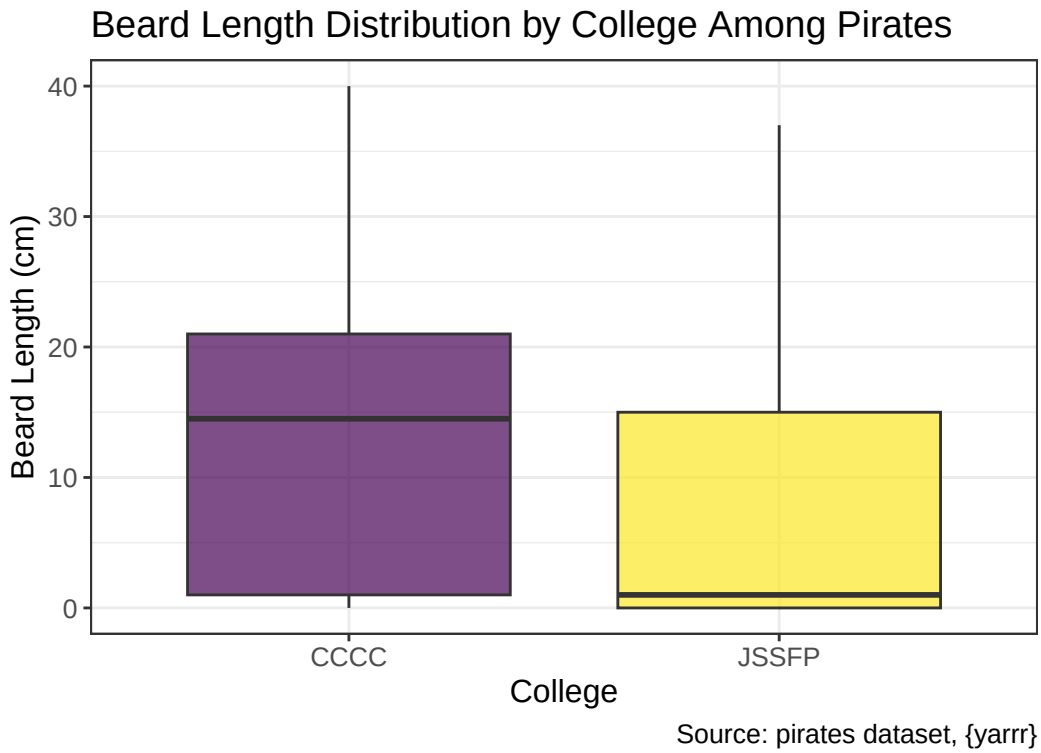
Beijing Daxing is a little different because it dips again in 2022 instead of steadily going up. That is likely because China kept stricter COVID restrictions for longer. After that it starts climbing again in 2023 and 2024. Overall, everything shows a steady recovery back toward normal levels.

2 Pirates Analysis

This section explores the pirates dataset from the `{yarrr}` package, focusing on how beard length varies across the two colleges represented in the data: CCCC and JSSFP.

2.1 Plot

Figure 2: Boxplots comparing beard length distributions between pirates who attended CCCC versus JSSFP (Phillips, 2017).



2.2 Narrative

Figure 2 shows a pretty clear difference between the two colleges. Pirates from CCCC usually have longer beards, and their values are more grouped together, which makes it seem like longer beards are the norm there.

Pirates from JSSFP are a lot more spread out. Their median is lower, but there is a bigger range overall. The long upper whisker shows that while most of them have shorter beards, a few grow really long ones.

It makes you wonder if something like college culture or different norms plays a role in how they keep their beards.

3 Monte Carlo Numerical Integration

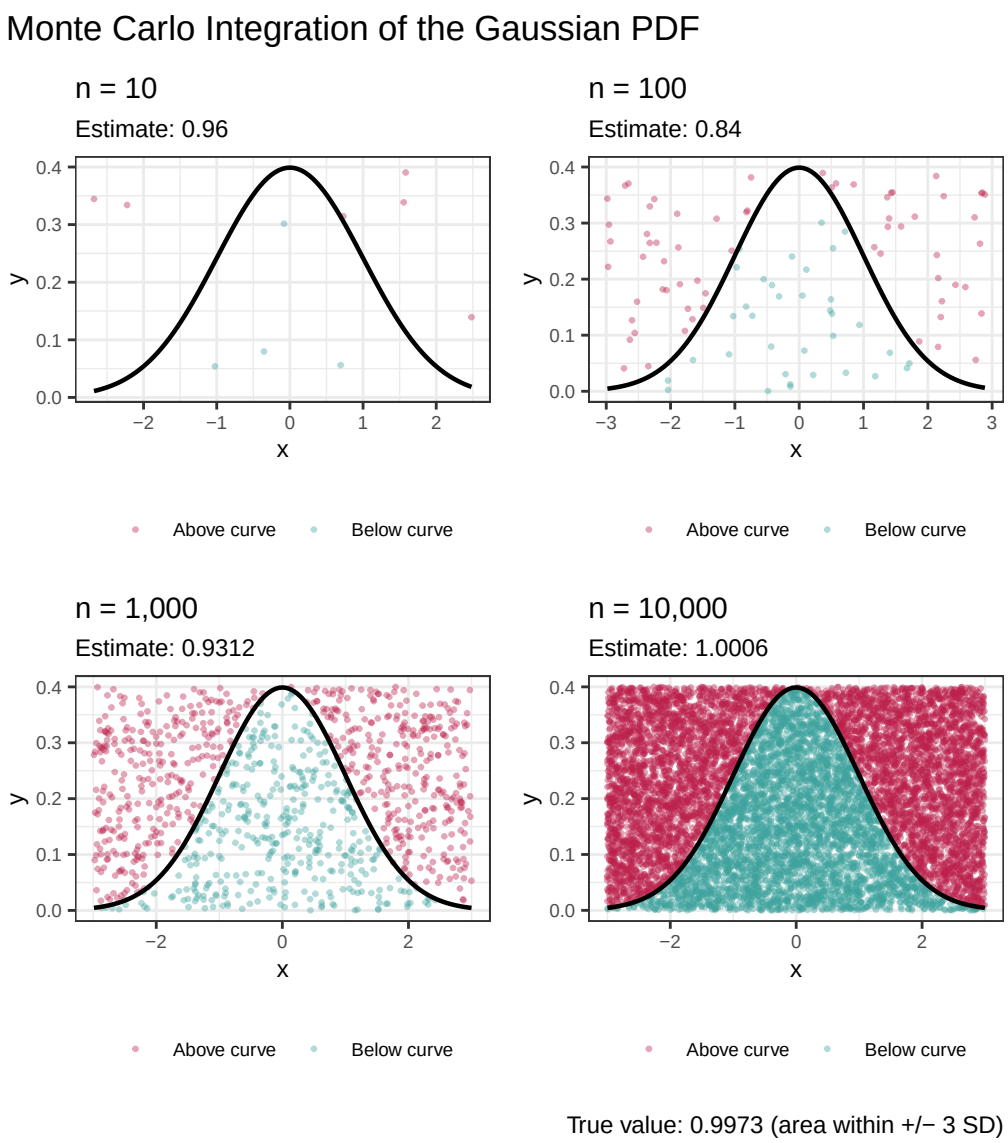
This section demonstrates the Monte Carlo approach to numerical integration applied to the Gaussian (Normal) probability density function with x bounds of -3 to 3 and y bounds of 0 to 0.4.

The Monte Carlo method works by randomly sampling points within a rectangular window and calculating the proportion that fall at or below the function. Multiplying that proportion by the

rectangle's area yields an estimate of the integral. The true value of the integral (the area under the standard normal curve from -3 to 3) is approximately 0.9973. This value equals $P(-3 \leq Z \leq 3)$ for a standard normal distribution.

3.1 Small Multiple Visualization

Figure 3: Monte Carlo integration of the Gaussian PDF at four sample sizes. As sample size increases, the area estimate converges toward the true value of approximately 0.9973.



3.2 Narrative

As the number of points increases, the estimate gets closer to the true integral value of about 0.9973. With only 10 points, it is pretty unreliable since there just is not enough data.

Once you get to 100 and 1,000 points, it starts getting better, and by 10,000 points it is really close to the true value. This is basically how Monte Carlo methods work. More random samples give you a better estimate.

You can also see it in Figure 3. With fewer points, everything looks scattered and does not match the curve very well. With 10,000 points, the space fills in more and the points under the curve line up much better with the actual area.

4 Planning and Prompting GenAI Tools

This section explores how the quality of a prompt to a generative AI tool affects the quality of its output, using the calcium dataset as a test case.

4.1 Data Context

The calcium dataset follows 31 women over four years. At baseline, researchers measured ulnar calcium levels in each woman's dominant arm. Fifteen women served as a null treatment group with no intervention; sixteen received dietary and exercise assistance throughout the study. Calcium measurements were taken annually (Initial, Year 1, Year 2, Year 3). The data as provided are not tidy. Columns represent repeated measurements at different time points, with the two groups occupying separate column blocks with no group identifier variable.

4.2 My Plan

Goal: I want to explore how ulnar calcium levels change over time and whether the diet/exercise treatment group differs from the null group.

Nouns: calcium.csv, tidyr, ggplot2, pivot_longer, treatment group variable, line plot

Verbs: load, tidy, pivot, mutate, plot

Steps:

1. Load calcium.csv and the packages tidyr and ggplot2.
2. The data is not tidy. Columns represent repeated ulnar calcium measurements at different time points and need to be pivoted from wide to long format.
3. Use pivot_longer() to reshape the time-point columns into a time variable and a calcium measurement value.
4. Add a treatment group variable that labels each participant as either the diet/exercise intervention group or the control group. Check whether a group indicator already exists in the raw data before pivoting and carry it through.
5. Create a line plot using ggplot2 showing ulnar calcium over time, with separate lines for each treatment group to assess whether the intervention group diverges from the control over time.

4.3 Plan-Informed Response

The following is the response generated by Anthropic Claude (Anthropic, 2026) when provided with the calcium.csv file and the plan above as a prompt on April 16, 2026.

Claude pushed back on the plan before generating code. It identified that the duplicate column names (y0, y1, y2, y3 appearing twice) meant pivot_longer() would not work cleanly without first splitting the data by column position. It also noted that there is no group indicator in the raw data. Group membership is encoded entirely by column position, with columns 1 through 4 representing the control group and columns 5 through 8 the diet/exercise group. Claude further identified that row 16 has missing values in the first four columns, confirming the groups are independent (15 control, 16 treatment) rather than paired measurements.

The full response text from Claude included: *"Pushing back before writing any code. Your plan has a problem. There is no group indicator in this data. The columns are duplicated. This is a structure where the first four columns are one group and the last four columns are the other. Group membership is encoded by column position, not by a variable. pivot_longer() alone won't work cleanly because you have duplicate column names."*

Claude provided corrected R code that split the data by column position before pivoting. No static visualization was produced by this prompt; the response consisted of corrected code and analytical commentary on the data structure.

4.4 Generic Response

The following was generated using a generic prompt ("Create a data visualization that will help me understand these data") with the same calcium.csv file submitted to Anthropic Claude on March 31, 2026.

The generic prompt produced a visualization showing group means with a toggle for individual lines. Claude noted that both groups follow the same pattern. Values stay stable across the first three waves then drop at the final measurement, and Group A starts higher but both groups converge to nearly identical final values (~64.8 and ~64.5).

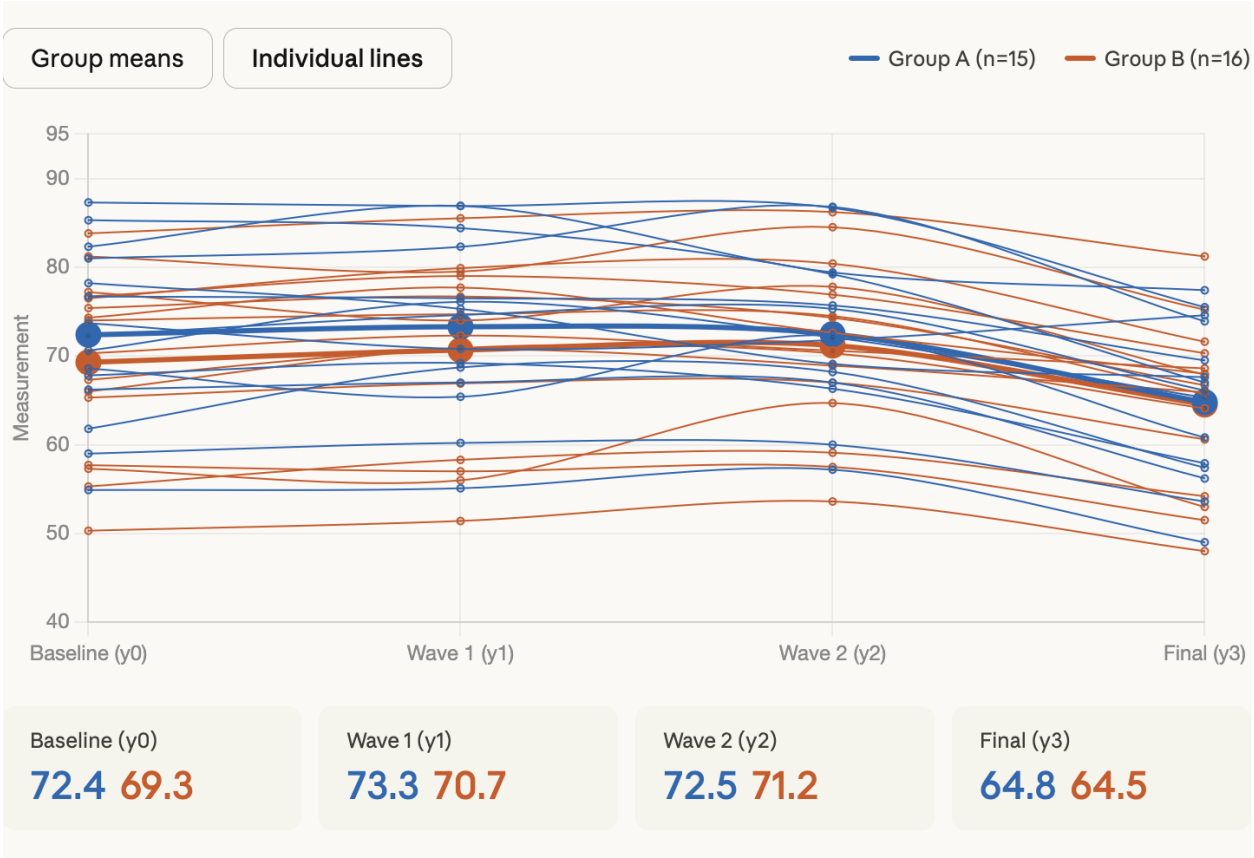
4.5 Comparison and Reflection

The difference between the two responses mostly comes down to how specific the prompt was. The generic prompt gave a nice looking graph, but it did not really go that deep. Since it did not have much context, it just assumed the data was already clean and described what it saw.

The plan based prompt worked a lot better because it gave more direction. It made Claude actually look at how the data was structured instead of just jumping into plotting. Because of that, it caught issues like duplicate column names and the missing group variable.

Instead of just making a quick visualization, it actually fixed the data and gave something more useful. It shows that better prompts do not just give better answers, they help catch problems earlier too.

Figure 4: Generic prompt Claude response: group means for ulnar calcium over four time points



5 Reflection

This course definitely changed how I think about data analysis. Before this, I mostly saw coding as just writing commands to get some output. Now I see that there is more of a process to doing it well.

The PCIP framework helped a lot with that. Instead of jumping straight into code, I now spend more time thinking about what I actually want to figure out and how to get there. It makes everything feel more intentional and easier to fix when something breaks.

Working with the tidyverse also helped me understand what clean data really looks like. The calcium dataset is a good example. It seems simple at first, but it is not useful until you reshape it properly. Learning how to use pivot_longer and work with long format data is something I will probably use a lot going forward.

ggplot2 also started to click more. Thinking about plots in terms of mappings and layers makes it feel more structured instead of just guessing what works.

Using Quarto showed me what reproducibility actually means. Having everything in one place from raw data to final output makes it way easier to follow and trust. It is a lot better than having separate scripts and screenshots all over the place, and it is something I want to keep using in the future.

References

Anthropic. (2026). *Claude sonnet 4.6*. https://www.anthropic.com

Phillips, N. (2017). *Yarrr: A companion to the e-book "YaRrr!: The pirate's guide to r"*. https://CRAN.R-project.org/package=yarrr

Wikipedia contributors. (2026). *List of busiest airports by passenger traffic*. https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic

GenAI Usage Appendix

5.1 Alt Text

Alt text for all figures in this document was written by the author and embedded directly as `#| fig-alt:` chunk options during document creation. No GenAI tools were used for alt text generation.

5.2 Planning and Prompting Section

Tool: Anthropic Claude Sonnet 4.6 **Date:** April 16, 2026

Plan-informed prompt: The calcium.csv file was uploaded alongside the full plan text documented in Section 4.2 of this document.

Full response: Included in Section 4.3 of this document.

Generic prompt: "Create a data visualization that will help me understand these data." [calcium.csv uploaded] **Date of generic prompt:** March 31, 2026 **Full response:** Included in Section 4.4 of this document.

Code Appendix

```r
library(tidyverse)
library(kableExtra)
library(yarrr)
library(patchwork)
# Build the data manually from Wikipedia
airports_raw <- tibble(
  Airport = rep(c("ATL", "DXB", "LHR", "DFW", "FRA", "PKX"), each = 5),
  Year = rep(2020:2024, times = 6),
  Passengers = c(
    # ATL
    42918685, 75704760, 93699630, 104653451, 106302208,
    # DXB
    25836771, 29110609, 66069981, 86994365, 95200000,
    # LHR
    22111469, 19408904, 61614508, 79183364, 84463000,
    # DFW
    39364990, 62465756, 73362946, 81755538, 85660127,
    # FRA
    18768601, 24812849, 48918482, 59355389, 63189666,
    # PKX
    25051012, 37916054, 34466946, 49441029, 53618949
  )
)

# Convert to millions for readability
airports_clean <- airports_raw %>%
  mutate(
    passengers_m = round(Passengers / 1e6, 2)
  )

# Pivot wide for table
airports_wide <- airports_clean %>%
  select(Airport, Year, passengers_m) %>%
  pivot_wider(
    names_from = Year,
    values_from = passengers_m
  )
airports_wide %>%
  kable(
    digits = 2,
    col.names = c("Airport (IATA)", "2020", "2021", "2022", "2023", "2024"),
    align = "lrrrrr"
  ) %>%
  kable_classic() %>%
```

```
  add_footnote(
    label = "Passenger counts sourced from Wikipedia: List of Busiest Airports by Passenger Tra
    notation = "none",
    threeparttable = TRUE
  )
ggplot(
  data = airports_clean,
  mapping = aes(
    x = Year,
    y = passengers_m,
    color = Airport,
    linetype = Airport,
    group = Airport
  )
) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 2) +
  scale_color_viridis_d() +
  scale_x_continuous(breaks = 2020:2024) +
  labs(
    x = "Year",
    y = "Total Passengers (Millions)",
    color = "Airport (IATA)",
    linetype = "Airport (IATA)",
    title = "Passenger Traffic at Six Major Airports, 2020-2024",
    caption = "Source: Wikipedia, List of Busiest Airports by Passenger Traffic"
  ) +
  theme_bw() +
  theme(
    text = element_text(size = 12),
    legend.position = "right"
  )
data(pirates)
ggplot(
  data = pirates,
  mapping = aes(
    x = college,
    y = beard.length,
    fill = college
  )
) +
  geom_boxplot(alpha = 0.7) +
  scale_fill_viridis_d() +
  labs(
    x = "College",
    y = "Beard Length (cm)",
    title = "Beard Length Distribution by College Among Pirates",
```

```r
    caption = "Source: pirates dataset, {yarrr}"
  ) +
  theme_bw() +
  theme(
    legend.position = "none",
    text = element_text(size = 12)
  )
# Monte Carlo simulation function
run_mc_integration <- function(n, x_min = -3, x_max = 3,
                               y_min = 0, y_max = 0.4) {
  # Sample random x and y coordinates
  x_vals <- runif(n, min = x_min, max = x_max)
  y_vals <- runif(n, min = y_min, max = y_max)

  # Evaluate Gaussian PDF at each x
  fx <- dnorm(x_vals, mean = 0, sd = 1)

  # Determine if point falls under the curve
  under_curve <- y_vals <= fx

  # Calculate area estimate
  rect_area <- (x_max - x_min) * (y_max - y_min)
  proportion <- mean(under_curve)
  estimate <- proportion * rect_area

  # Return results as data frame
  tibble(
    x = x_vals,
    y = y_vals,
    under = under_curve,
    n = n,
    estimate = round(estimate, 4)
  )
}

# Create one plot for a given sample size
make_mc_plot <- function(n) {
  df <- run_mc_integration(n)
  est <- unique(df$estimate)

  ggplot(df, aes(x = x, y = y, color = under)) +
    geom_point(alpha = 0.4, size = 0.5) +
    stat_function(
      fun = dnorm,
      args = list(mean = 0, sd = 1),
      color = "black",
      linewidth = 0.8
```

```r
    ) +
    scale_color_manual(
      values = c("TRUE" = "#3EA39E", "FALSE" = "#BC204B"),
      labels = c("TRUE" = "Below curve", "FALSE" = "Above curve")
    ) +
    labs(
      title = paste0("n = ", format(n, big.mark = ",")),
      subtitle = paste0("Estimate: ", est),
      x = "x",
      y = "y",
      color = NULL
    ) +
    theme_bw() +
    theme(
      text = element_text(size = 9),
      legend.position = "bottom"
    )
}
set.seed(184)

plot_10    <- make_mc_plot(10)
plot_100   <- make_mc_plot(100)
plot_1000  <- make_mc_plot(1000)
plot_10000 <- make_mc_plot(10000)

# Combine panels with patchwork
plot_10 + plot_100 + plot_1000 + plot_10000 +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Monte Carlo Integration of the Gaussian PDF",
    caption = "True value: 0.9973 (area within +/- 3 SD)"
  )
```